LLM ENGINEERING

Bonus chapter 2

Bonus: Long-Context Positional Extension

Self-paced reading

Interactive version: https://llm.strueber.dev/#/bonus-02-long-context

Bethgelab · University of Tübingen · Winter term 2026/27

Text and figures under CC BY 4.0, code under the MIT licence.

The problem

1

RoPE has no maximum position.

Why not serve an 8K-trained model at 32K?

Answer A1

Each pair turns at its own rate

(1) $$\omega_j = b^{-2j/d_r}, \qquad \text{angle at position } m = m\,\omega_j, \qquad \Delta = n - m$$

- `ρ = L_t / L_0`: target over trained length
- Fast pairs: short wavelength, many turns
- Slow pairs: long wavelength, few turns
- Every method below reshapes these bands

▼ **Pairs, frequencies and the extension ratio**

Formula (1) gives the rate at which each pair turns. Each head has an even number `d_r` of rotated coordinates. They form `d_r/2` pairs, indexed `j = 0, …, d_r/2 - 1`, and each pair rotates as a two-dimensional vector. Pair `j` turns at `ω_j = b^(-2j/d_r)` radians per token, so at position `m` its angle is `m·ω_j`. The base `b` is not itself a frequency. It sets every pair's frequency through this formula. Code calls `ω_j` the inverse frequency because it is the reciprocal of `b^(2j/d_r)`.

Week 3 showed that the rotary part of the score between a query at position `m` and a key at position `n` depends on the two positions only through the signed offset `Δ = n - m`. In causal attention a query sees only keys at or before its own position, so `n ≤ m` and `Δ ≤ 0`. Because the offsets are zero or negative, "larger" is ambiguous: write a range as a signed interval, such as `-(L_0 - 1) ≤ Δ ≤ 0` within training, or bound `|Δ|`.

The extension ratio `ρ = L_t / L_0 ≥ 1` divides the target length by the trained length. Serving an 8K-trained model at 32K means `ρ = 4`.

Write `D_h` for the head dimension. With full rotary, `d_r = D_h` and every coordinate of the head rotates. With partial rotary, `D_h - d_r` coordinates stay unrotated. An attention logit sums over the coordinates of the head, so with partial rotary it splits into a part from the rotated coordinates and a part from the unrotated ones.

The frequency falls as `j` grows. Low-index pairs rotate quickly: their wavelength is short, and the same offset changes their angle a lot. Within the training length they complete many turns. High-index pairs rotate slowly: their wavelength is long, the same offset changes their angle little, and they complete few turns. Every method in this chapter changes how these bands behave when `L_t > L_0`.

PI and NTK

PI divides every angle by ρ

$$m \mapsto \frac{m}{\rho}, \qquad \Delta \mapsto \frac{\Delta}{\rho}, \qquad \omega_j \mapsto \frac{\omega_j}{\rho} \tag{2}$$

- **Position Interpolation** (PI) (Chen et al., 2023)
- Adjacent angle: one extra token's rotation
- Position 32,767 maps to 8,191.75
- Fast pairs lose resolution too (Peng et al., 2024)

▼ Rescaling positions rescales every frequency

Position Interpolation (PI) makes a long input look like a short one (Chen et al., 2023). It divides every position by `ρ`, so position `m` becomes `m/ρ` and every offset `Δ` becomes `Δ/ρ` (Formula 2). A 32K input then spans the same range of positions that an 8K input spanned in training.

Under PI the query is at `m/ρ` and the key at `n/ρ`, so pair `j` sees the signed offset `(n - m)/ρ = Δ/ρ`, and its relative angle is `Δ·ω_j/ρ` instead of `Δ·ω_j`. The angle at a single position shows the same thing: `(m/ρ)·ω_j = m·(ω_j/ρ)`. Rescaling the positions is therefore the same as rescaling the frequencies, and PI can be written as `ω_j ↦ ω_j/ρ`, the third map in Formula (2). We call `ω_j/ρ` the PI frequency of pair `j`.

The **adjacent angle** of pair `j` is the angle that one extra token adds to its rotation, which is `ω_j` itself. To compare a remapping with the original, divide the new adjacent angle `ω'_j` by the old one. This adjacent-angle ratio `ω'_j/ω_j` is `1/ρ` for every pair under PI: each pair turns `ρ` times more slowly per token than it did in training.

Take the illustrative values `L_0 = 8,192`, `L_t = 32,768` and `ρ = 4`. The last target index, 32,767, maps to 32,767/4 = 8,191.75. That value lies inside the half-open range `[0, 8192)` that the trained length spans, but just above 8,191, the largest integer index training contained. So the largest target offset maps back into the original range of offsets, up to that fraction. Aligning the integer endpoints exactly would divide by `(L_t - 1)/(L_0 - 1)` instead. We use the nominal ratio `ρ = L_t/L_0`.

Chen et al. derive a stability bound for interpolation. In their LLaMA setup, they fine-tune each remapped model briefly, then check loss, retrieval, long-document tasks and quality within the original window (Chen et al., 2023).

Dividing every adjacent angle by `ρ` also has a cost. The fast pairs change their angle a lot from one token to the next, and the model uses them for fine distinctions between nearby

positions. PI shrinks their adjacent angle by the same factor as the slow pairs', so they lose resolution too. This diagnosis in terms of frequencies comes from the later YaRN analysis, not from PI's original theorem (Peng et al., 2024).

Static NTK-aware scaling raises the base

$$b' = b\,\rho^{d_r/(d_r-2)} \tag{3}$$

- Named after the neural tangent kernel (NTK)
- Static: one `ρ`, one base (bloc97, 2023)
- Dynamic NTK: recompute the base as length grows
- Adjusted base frequency (ABF): train a larger base

▼ **Three ways to change the base**

A second family of methods leaves the positions alone and changes the base. Static NTK-aware scaling, from 2023, takes its name from the neural tangent kernel (NTK), through an argument we do not derive. It picks one fixed target ratio `ρ` and replaces the base `b` by `b' = b·ρ^(d_r/(d_r-2))` (Formula 3) (bloc97, 2023). The formula first appeared in a community post. Maintained libraries implement it, and their code shows how it is computed.

To see what the new base does to each pair, put `b'` into Formula (1): `(b')^(-2j/d_r) = b^(-2j/d_r)·ρ^(-2j/(d_r-2)) = ω_j·ρ^(-2j/(d_r-2))`. The adjacent-angle ratio is therefore `ω'_j/ω_j = ρ^(-2j/(d_r-2))`. At `j = 0` the exponent is 0 and the ratio is 1, so the fastest pair keeps its angle. At the slowest pair, `j = d_r/2 - 1 = (d_r - 2)/2`, the exponent is −1 and the ratio is `1/ρ`, the same as under PI. Between the two ends the ratio falls by the same factor `ρ^(-2/(d_r-2))` from each pair to the next. For the illustrative case `ρ = 4` and `d_r = 64`, the new base is `b' = b·4^(64/62)`, about 4.183 times the old one.

Static scaling fixes `ρ` and `b'` once, so it applies its full remapping at every length, short inputs included: a 1K prompt gets the same remapped frequencies as a 32K one. Dynamic NTK recomputes a base from the observed sequence length as that length grows, so it remaps only as far as the current input needs. The recomputation differs between variants, and Hugging Face documents the variants side by side (Hugging Face, n.d.).

Recomputing the base during generation adds a constraint on the KV cache. The cached keys were rotated under an earlier base, so the implementation needs a policy that keeps them compatible with the current one. It can recompute or re-rotate the cached keys, or it can generate with one fixed base.

A trained high base, called adjusted base frequency (ABF), changes the same knob differently. You pick a larger `b` and train, or continue training, with it, so the weights adapt to the new base. ABF shares the knob with static NTK-aware scaling and Dynamic NTK, but not their algorithm.

Because all three methods change the base, a report that says only "the base was raised" does not say which method was used. A reproduction record names the variant, the original context, the target ratio, `d_r`, the base update policy, the cache policy and the training history.

5

What ratios do PI and NTK give?

$$\rho = 4,\ d_r = 64$$

$$\text{fastest pair } j = 0,\ \text{slowest pair } j = 31$$

Answer A2

Compare PI and NTK pair by pair

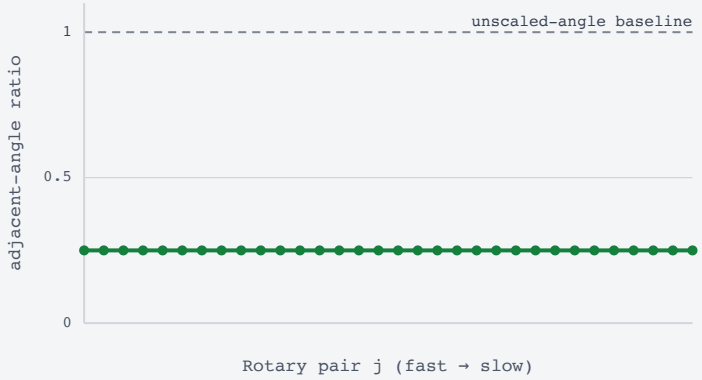
- Each dot: one rotary pair
- Height: new angle over original angle

Adjacent-angle ratio

d_r: 32 **64** ρ: 2x **4x** 8x

method: No extension **Position Interpolation** NTK-aware

L_0=8K, L_t=32K, ρ=4: ratio by rotary pair



Select a rotary-pair dot to read its adjacent-angle ratio.

PI divides every adjacent angle by ρ. Static NTK-aware scaling leaves the fastest pair unchanged and moves the others step by step toward 1/ρ. Besides the No extension reference, the widget plots these two methods only. A ratio of 1 means a pair turns as in training; it says nothing about the length the model accepts or uses well. The curves are derived from the formulas; they are not measurements of model quality.

▼ Reading the adjacent-angle ratio

The widget plots the adjacent-angle ratio of every rotary pair. Each dot is one pair, from the fastest, `j = 0`, on the left to the slowest on the right. Its height is the pair's new adjacent angle divided by its original one, so a height of 1 means the pair turns as it did in training. Select a dot to read its ratio.

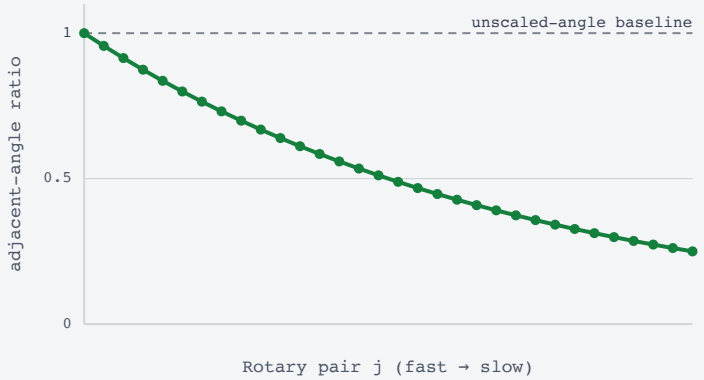
Compare the two methods at both ends of the axis and in between, with No extension as the reference. PI draws a flat line at `1/ρ`. NTK-aware scaling starts at 1 and falls to `1/ρ` at the last dot. A larger `ρ` lowers PI's whole line and NTK-aware scaling's slow end, while NTK-aware scaling's fast end stays at 1. Changing `d_r` changes only the number of dots. The NTK-aware ratio `ρ^(-2j/(d_r-2))` can be written as `ρ^(-j/(d_r/2 - 1))`. The ratio depends only on how far along the axis the pair is, `j/(d_r/2 - 1)`, so the curve keeps its shape whatever `d_r` is.

Adjacent-angle ratio

d_r: 32 64 ρ: 2x 4x 8x

method: No extension Position Interpolation NTK-aware

L_0=8K, L_t=32K, ρ=4: ratio by rotary pair



Select a rotary-pair dot to read its adjacent-angle ratio.

PI divides every adjacent angle by ρ. Static NTK-aware scaling leaves the fastest pair unchanged and moves the others step by step toward 1/ρ. Besides the No extension reference, the widget plots these two methods only. A ratio of 1 means a pair turns as in training; it says nothing about the length the model accepts or uses well. The curves are derived from the formulas; they are not measurements of model quality.

NTK-aware scaling at ρ = 4: the fastest pair keeps a ratio of 1, and the ratio falls to 0.25 at the slowest pair.

YaRN and LongRoPE

7

YaRN blends by rotation count

(4)
$$\lambda_j = \frac{2\pi}{\omega_j} = 2\pi b^{2j/d_r}, \qquad r_j = \frac{L_0}{\lambda_j}$$

(5)
$$\omega'_j = (1-\gamma_j)\frac{\omega_j}{\rho} + \gamma_j\,\omega_j$$

- **YaRN** (Peng et al., 2024)
- Few rotations: interpolate like PI
- Many rotations: keep the original frequency
- In between: blend, called NTK-by-parts

▼ **From rotation counts to the blend**

YaRN treats the frequency bands selectively (Peng et al., 2024). Its rule needs two quantities for each pair. The first is the wavelength `λ_j = 2π/ω_j`. Because `ω_j` is measured in radians per token and one full turn is `2π` radians, `λ_j` is measured in tokens per cycle: the number of tokens pair `j` takes for one turn. With `ω_j = b^(-2j/d_r)` this gives `λ_j = 2π·b^(2j/d_r)` (Formula 4), so the wavelength grows with `j`.

The second quantity is the number of rotations the pair completes over the original context, `r_j = L_0/λ_j`. This is the paper's nominal count. Counting only the span between the first and the last position would give `(L_0 - 1)/λ_j`, the number of cycles between the two endpoints. We keep the nominal count, as the paper does.

A ramp turns the rotation count into a blending weight:

$$\gamma_j = \mathrm{clip}\left(\frac{r_j - \alpha}{\beta - \alpha},\, 0,\, 1\right)$$

The weight is 0 for a pair with fewer than `α` rotations and 1 for a pair with more than `β`, and it rises linearly between the two. It goes from 0 for the slow pairs, which have long wavelengths and few rotations, to 1 for the fast pairs, which have short wavelengths and many rotations. For the Llama family the paper recommends `α = 1` and `β = 32` (Peng et al., 2024). The authors' reference code calls the same two values `beta_slow` and `beta_fast` (Quesnelle et al., 2023). Both are tunable choices made for that family.

Formula (5) uses the weight to mix two frequencies. At `γ_j = 0` it gives `ω_j/ρ`, the PI frequency of pair `j`. At `γ_j = 1` it gives the original `ω_j`. For a weight between 0 and 1, the coefficients `1 - γ_j` and `γ_j` are nonnegative and add up to 1, so `ω'_j` is a convex blend of the two frequencies and lies between them. The slow pairs are therefore interpolated as under PI, the fast pairs keep their original frequency, and the band between them is blended. YaRN's paper calls this selective blend NTK-by-parts interpolation. It is the step between the earlier methods and YaRN, which adds an attention scaling to it.

8

Which pairs does YaRN keep?

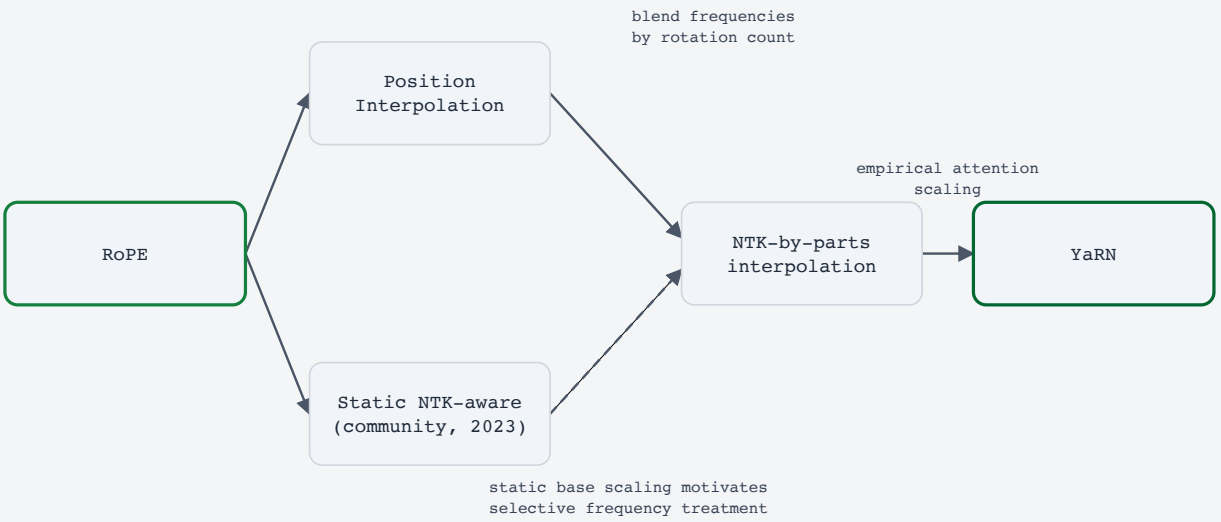
What is γ for pair j = 16?
L_0 = 8K, b = 10,000, d_r = 64
ramp α = 1, β = 32

Answer A3

YaRN adds attention scaling to the blend

$$a(\rho) = 0.1 \ln \rho + 1 \tag{6}$$

- Fitted on Llama models (Peng et al., 2024)
- Scales Q and K by `a`
- Counters entropy growth with more keys



Schematic after Peng et al. (ICLR 2024), Figure 1.

A solid edge carries a mechanism into the next method; the dashed edge carries a motivating idea only. The edges show which idea each method takes from another, not that the methods are equivalent or that one inherits from another.

▼ Attention scaling and the method family

Remapping the frequencies is one half of YaRN. The other half is an attention scaling. Peng et al. fitted it on full-rotary Llama models, picking for each ratio the factor with the lowest perplexity (Peng et al., 2024). For the extension ratio `ρ`, the factor is `a(ρ) = 0.1 ln ρ + 1` (Formula 6), and YaRN multiplies both the query and the key by it. A logit is the dot product of a query and a key, so with full rotary every logit is multiplied by `a²`. For `ρ > 1` the factor exceeds 1, and it grows slowly with `ρ`.

Peng et al. link the factor to the increased entropy of attention at long inputs. As more keys enter each softmax, the attention weight spreads over more of them, and the entropy of the attention distribution grows. Multiplying every logit by a factor above 1 sharpens the softmax and lowers that entropy again. Check the formula and its constants on your own model before you reuse them. The authors' reference implementation shows how the ramp and the scaling are applied in code (Quesnelle et al., 2023).

The lineage figure, redrawn after Figure 1 of Peng et al., shows how YaRN grew out of the earlier methods (Peng et al., 2024). It has five nodes. RoPE branches into Position Interpolation and static NTK-aware scaling, and both lead into NTK-by-parts interpolation, the blend by rotation count. The edge from PI brings the mechanism: NTK-by-parts blends PI's interpolated frequencies with the original ones by rotation count. The dashed edge from static NTK-aware scaling brings only the idea that base scaling already treats the frequencies selectively, since it spares the fastest pair. The final edge, from NTK-by-parts to YaRN, adds the attention scaling.

Models apply YaRN to different layers, parts of the attention and training stages. The Training tab's recipe table lists them.

10

What is the new logit?

$\rho = 4$ gives $a \approx 1.1386$
$\ell_r = 2$ rotated, $\ell_u = 3$ unrotated
scale only the rotated Q and K

Answer A4

LongRoPE searches the rescale factors

- **LongRoPE** (Ding et al., 2024)
- Searched: one rescale factor per pair
- Searched: where rescaling starts
- Fine-tune to 256K, then search to 2,048K
- Readjusted for short contexts
- Used by the 128K Phi models (Abdin et al., 2024) (Microsoft et al., 2025)

▼ **Searching instead of a closed form**

PI, NTK-aware scaling and YaRN each compute the new frequencies from a closed-form rule. LongRoPE searches for them instead, guided by perplexity (Ding et al., 2024). It searches two non-uniformities. The first is one rescale factor for each rotary pair, so every pair can be slowed by its own amount. The second is a starting-position threshold: positions before it keep their original angles, and later positions get the searched rescaling. We do not derive the search.

The paper extends in stages. For LLaMA2 it first fine-tunes for 400 steps with the rescale factors searched for 128K. It then replaces them with the factors for 256K and fine-tunes for 600 more steps. A second search on the fine-tuned 256K model reaches 2,048K without further fine-tuning. The paper reports two such 2,048K models: this one, whose weights were adapted up to 256K, and one adapted only at 128K. Finally, it searches once more to readjust the rescale factors for short contexts, for example 4K and 8K. The search over both non-uniformities, the staged extension and this short-context readjustment set LongRoPE apart from the closed-form methods.

The schedule separates two lengths: the length used to adapt the weights, at most 256K, and the much larger length reached by remapping, 2,048K. A low perplexity in the search or a nominal maximum length is not evidence that the model uses its context well. Retrieval, tasks that need distant evidence, checks within the original window and serving measurements decide that.

Among Microsoft's Phi models, only the long-context (128K) ones use LongRoPE. The Phi-3 report extends phi-3-mini to 128K with it, and the Phi-3.5 models use it to go from 4K to 128K (Abdin et al., 2024). The Phi-4-mini models support 128K through LongRoPE as well (Microsoft et al., 2025).

Training

12

Three interventions, three costs

Intervention	Question it answers	Main cost
Inference-only remapping	Serve longer without training?	None up front; must pass the matrix
Continued long-context training	Can you afford to adapt the weights?	Training compute and long data
Target-length training	Must weights see the full length?	Attention, memory, batch and data

▼ Three interventions answer three questions

The remappings of the previous tabs change the angles. Reaching a longer context can also involve the weights, and three interventions answer three different questions.

Inference-only remapping asks whether you can serve longer inputs without any training. It changes the positional coordinates with one of the remappings above, such as PI, NTK-aware scaling or YaRN, and leaves the weights as they are. It costs nothing up front, so it is the cheapest intervention to try. It is acceptable only if the model passes the complete evaluation matrix. The matrix is a set of checks across lengths, evidence positions, tasks that need distant evidence, the original window and serving, and you declare its pass threshold before the run. The Evaluation tab sets out the matrix.

Continued long-context training asks whether you can afford to adapt the weights. It trains the checkpoint further on longer sequences. This costs training compute and needs long training data, and in return the model gets a chance to learn to use distant evidence.

Target-length training asks whether the weights must see the full target length during training. In the chosen training stage it exposes them to sequences of the full target length, and it pays what such sequences cost: attention work, memory, a smaller batch and enough long data.

The three names describe what you do to the model. They do not tell you the compute, memory, data or quality cost that a given run ends up with. Compare those outcomes with the same evaluation matrix.

Each intervention fits a different situation. Remapping alone fits when training is unavailable and the model already meets the declared threshold on the complete matrix. Short continued training fits when the weights must adapt and representative long data is available. Target-

length training fits when the weights must see the full target length and its compute, memory, batch and data costs are acceptable.

Real recipes mix training and remapping

Model	Training for length	Remapping
Llama 3 (Grattafiori et al., 2024)	Base 500,000 from the start; 405B: six stages, 8K to 128K	None in the report; 3.1 weights rescale by parts (Meta, 2025)
Qwen3 (Yang et al., 2025)	ABF to 1,000,000; a 32K stage	YaRN and Dual Chunk Attention (DCA) at inference
OLMo 3 (Ettinger et al., 2025)	8K to 65K on 50B to 100B tokens	YaRN on full-attention layers only
DeepSeek-V3 (DeepSeek-AI et al., 2024)	Two 1,000-step phases, 4K to 32K to 128K	YaRN on the RoPE part only
Long-context Phi (Abdin et al., 2024)	Extended to 128K	LongRoPE

▼ What each recipe teaches

The table lists five recipes from model reports. Each row pairs what the model trains on to reach its length with the remapping it uses, if any.

Llama 3 sets the RoPE base to 500,000 from the start of pre-training (Grattafiori et al., 2024). The report writes the base as θ. It is the course's b. For the 405B model the report then increases the context from the original 8K to 128K in six stages, over about 800B training tokens, so the weights see the full target length. The report mentions no YaRN. The report's models were released as Llama 3.1, and Meta's reference code for those weights rescales the frequencies by parts. The code uses the rotation count $r_j = L_0/\lambda_j$ of slide 7, over the original 8,192 tokens. Pairs with fewer than one rotation have their frequency divided by 8. Pairs with more than four rotations keep their frequency, and the pairs between are blended (Meta, 2025). The factor and both ramp boundaries come from a grid search, as a comment in the code says. The factor is 8, smaller than the extension ratio $L_t/L_0 = 16$. This is the mechanism of NTK-by-parts, with its own ramp boundaries and without YaRN's attention scaling.

Qwen3 raises its base from 10,000 to 1,000,000 with adjusted base frequency (ABF), the trained high base of slide 4, for a long-context pre-training stage at 32,768 tokens (Yang et al., 2025). So raising the base covers two different things: static NTK-aware scaling, a formula applied without training, and ABF, where you pick a large base and train with it. At inference Qwen3 adds YaRN and Dual Chunk Attention (DCA), a method that splits a long input into chunks for attention. Together they let Qwen3 serve inputs four times longer than its 32K training length (Yang et al., 2025).

OLMo 3 extends its context from 8,192 to 65,536 tokens in a training stage of 50B tokens for the 7B model and 100B tokens for the 32B model (Ettinger et al., 2025). During that stage it applies YaRN only to its full-attention layers. Three of every four layers use sliding-window attention, in which each token attends only to a fixed window of recent tokens. Those layers never see an offset beyond their window, however long the input, and OLMo 3 leaves their RoPE unchanged (Ettinger et al., 2025). Week 4 covers sliding windows.

DeepSeek-V3 extends from 4K to 32K and then to 128K in two training phases of 1,000 steps each, with YaRN applied in both (DeepSeek-AI et al., 2024). YaRN remaps only the RoPE part of its multi-head latent attention, the 64 rotated coordinates of the split described on slide 10.

The long-context Phi models, among them phi-3-mini-128K, the Phi-3.5 models and Phi-4-mini, reach 128K with LongRoPE, the searched rescaling of slide 11 (Abdin et al., 2024) (Microsoft et al., 2025).

The rows also differ in when they remap. Among the three YaRN rows, only Qwen3 applies it at inference alone. OLMo 3 and DeepSeek-V3 apply it during their extension training. Of these five, three use YaRN, one LongRoPE, and one relies on a high base, with by-parts rescaling in its released weights. No row is a default for every model.

Remapping keeps angles in range; training improves use

- Remapping: angles back in the trained range
- Remapping alone: no training on distant evidence
- More extension tokens improved OLMo 3 (Ettinger et al., 2025)
- PI adapted briefly after remapping (Chen et al., 2023)
- Longer sequences: more attention work and memory

▼ **Remapping and adaptation do different jobs**

A remapping changes only the position indices, or equivalently the frequencies, so that the angles of a long input stay in the range training produced. The model's internal states at the new lengths can still differ from anything it saw in training, and the remapping teaches it nothing about using distant tokens.

Adapting the weights on long data gives the model practice with distant tokens, and in the reports it improved long-context scores. OLMo 3 compared several token budgets for its extension stage. More tokens raised its scores on the long-context benchmark RULER, especially at longer sequence lengths (Ettinger et al., 2025). Position Interpolation, too, comes with adaptation: Chen et al. fine-tune briefly after interpolating (Chen et al., 2023).

These recipes worked for their own models. If your own measurement disagrees with what they suggest, trust the measurement.

Training on longer sequences also costs more. Longer sequences raise the attention work and the memory pressure of each step. The utilization a run actually reaches depends on the kernels, the parallelism, how sequences are packed and the hardware.

Evaluation

15

Context length is not context use

Check	Concrete 8K to 32K design
Language modeling	Negative log-likelihood (NLL) at 8K, 16K, 24K, 32K
Retrieval	Evidence at early, middle and late positions
Distant-evidence use	Question answering (QA), aggregation or summaries needing that evidence
Original window and serving	8K loss and tasks, short prompts, latency, memory
Robustness	Several prompts and documents; seeds or intervals
Reproducibility	Tokenizer, effective target, prompt format, decoding
Decision	Task floor and allowed regression, set in advance

▼ **Seven checks before calling an extension done**

Slide 12 asked every intervention to pass one evaluation matrix. The full matrix has seven rows, and the table makes each one concrete for an extension from `L_0 = 8K` to `L_t = 32K`. A claim that the extended model works at 32K is credible only when it covers all seven.

Language modeling is measured as the negative log-likelihood (NLL) of held-out text: the average of `-log p` over the probabilities the model gives each actual next token. Report it at 8K, 16K, 24K and 32K tokens, so you can see whether prediction gets worse as the input grows. Its exponential, the perplexity, carries the same information and is optional.

Retrieval measures accuracy when the evidence an answer needs is placed early, in the middle or late in the input. Slide 1 noted that how badly a model fails depends on where the evidence appears, so each position is reported on its own.

Finding a string is one skill, and using what you found is another. The distant-evidence row therefore asks for at least one task whose answer requires the distant evidence: question answering (QA), aggregation or summarization over the long input.

The original-window and serving row rechecks the extended model where it already worked. Measure the loss and the tasks at 8K, the quality on short prompts, and the latency and memory of serving. A remapping can change the model's behavior on short inputs too: static NTK-aware scaling, for one, applies its full remapping at every length.

Robustness means several prompts and several documents, plus repeated seeds or confidence intervals, so that one prompt or one document does not decide the result. Reproducibility means recording the tokenizer, the effective target length the inputs reach in tokens, the prompt format and the decoding details.

The decision row states two numbers before the comparison runs: the floor the target task must reach, and the largest short-context or serving regression you will accept. A threshold chosen after the results are known can be fitted to them.

A nominal maximum context or a flat loss curve is not evidence of useful context on its own. Whichever intervention you choose, record the serving latency and memory, the regression in the original window, the quality on the target task, and the remaining risk that inputs the model accepts at 32K still fail to use distant evidence.

The formulas fix what each closed-form method does to each pair: the ratios of PI, NTK-aware scaling and YaRN are algebra, and LongRoPE's factors come from a search. Whether the extended model works is a different question. The attention-scaling constants, how well the model adapts on long data and its final quality can only be measured, and the matrix is where you measure them.

16

What is still missing?

8K to 32K: flat perplexity at 32K
retrieval passes early, middle and late

Answer A5

17

So why not serve an 8K-trained model at 32K?

Answer A6

Six results on long context

- Past `L_0`, slow pairs meet unseen angles
- PI scales all pairs; NTK spares the fastest
- YaRN: blend by rotation count, scale attention
- LongRoPE: searched per-pair factors
- Recipes mix training and remapping
- Test context use, not context length

▼ Six results

Past the trained length `L_0`, RoPE still gives every position an angle, but the slow pairs reach angles that training never produced ("Slow pairs leave the trained range").

Position Interpolation divides the adjacent angle of every pair by `ρ`, while static NTK-aware scaling raises the base, keeps the fastest pair's angle and brings the slowest pair to `1/ρ` of its angle ("PI divides every angle by ρ", "Static NTK-aware scaling raises the base"). Dynamic NTK recomputes the base as the input grows, and ABF trains with a larger base ("Static NTK-aware scaling raises the base").

YaRN keeps the frequencies of pairs with many rotations, interpolates pairs with few, blends the band between them and scales attention by a fitted factor ("YaRN blends by rotation count", "YaRN adds attention scaling to the blend").

Guided by perplexity, LongRoPE searches a rescale factor for each pair and the position where rescaling starts, fine-tunes in stages to 256K, searches again to reach 2,048K, and is used by the 128K Phi models ("LongRoPE searches the rescale factors").

Inference-only remapping, continued long-context training and target-length training answer different questions at different costs ("Three interventions, three costs"). Real recipes combine training on long data with a remapping or a high base: remapping keeps the angles in range, and training improves the use of distant tokens ("Real recipes mix training and remapping", "Remapping keeps angles in range; training improves use").

Only an evaluation of context use against a threshold set in advance shows whether an extension works ("Context length is not context use").

Reflection questions

No submission. Bring short answers to the Week 4 tutorial.

1. Take `b = 10,000`, `d_r = 64` and an 8K-trained model served at 32K, so `ρ = 4`. Under static NTK-aware scaling, which of the seven pairs that never complete a turn within 8K still go beyond their largest trained angle at 32K, and by what factor does each exceed it? What do PI and YaRN do to the same seven pairs?
2. Your course model uses full-head RoPE with `b = 10,000` and `D_h = 128`, trained on 1,024-token windows. It must now answer questions about 4,096-token documents, and you can afford one short continued-training run. Choose how to change the rotary frequencies and with which settings, say what the run trains on, and state the evaluation and the acceptance threshold you would fix before the run.
3. An 8K-trained model with full rotary, `b = 10,000` and `d_r = 64` must serve inputs of up to 32K tokens without any training, but most of its prompts are under 2K tokens. Compare PI and YaRN, both at `ρ = 4`: which pairs and which logits of a 1K prompt does each change, and by how much? Which would you deploy, and what must you measure before you decide?

References

RoPE and extension methods

- bloc97 (2023). NTK-Aware Scaled RoPE allows LLaMA models to have extended (8k+) context size without any fine-tuning and minimal perplexity degradation. Community post.
- Chen, S., Wong, S., Chen, L., & Tian, Y. (2023). Extending Context Window of Large Language Models via Positional Interpolation. arXiv preprint arXiv:2306.15595.
- Ding, Y., Zhang, L. L., Zhang, C., Xu, Y., Shang, N., Xu, J., Yang, F., & Yang, M. (2024). LongRoPE: Extending LLM Context Window Beyond 2 Million Tokens. *ICML*, PMLR 235, 11091-11104.
- Gelberg, Y., Eguchi, K., Akiba, T., & Cetin, E. (2026). Extending the Context of Pretrained LLMs by Dropping Their Positional Embedding. *ICLR*.
- Peng, B., Quesnelle, J., Fan, H., & Shippole, E. (2024). YaRN: Efficient Context Window Extension of Large Language Models. *ICLR*.
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., & Liu, Y. (2024). RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing*, 568, 127063.

Model reports

- Abdin, M., Aneja, J., Awadalla, H., et al. (2024). Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv preprint arXiv:2404.14219.
- DeepSeek-AI, Liu, A., Feng, B., et al. (2024). DeepSeek-V3 Technical Report. arXiv preprint arXiv:2412.19437.
- Ettinger, A., Bertsch, A., Kuehl, B., et al. (2025). Olmo 3. arXiv preprint arXiv:2512.13961.
- Grattafiori, A., Dubey, A., Jauhri, A., et al. (2024). The Llama 3 Herd of Models. arXiv preprint arXiv:2407.21783.
- Merrill, W., Li, Y., Romero, T., et al. (2026). Olmo Hybrid: From Theory to Practice and Back. arXiv preprint arXiv:2604.03444.
- Microsoft, Abouelenin, A., Ashfaq, A., et al. (2025). Phi-4-Mini Technical Report: Compact yet Powerful Multimodal Language Models via Mixture-of-LoRAs. arXiv preprint arXiv:2503.01743.
- The Microsoft AI Team (2026). MAI-Thinking-1: Building a Hill-Climbing Machine. Technical report, Microsoft AI.
- Yang, A., Li, A., Yang, B., et al. (2025). Qwen3 Technical Report. arXiv preprint arXiv:2505.09388.

22

Implementation sources

- DeepSeek-AI. (2025). DeepSeek-V3 inference code (inference/model.py).
- Hugging Face. (n.d.). Utilities for Rotary Embedding.
- Meta. (2025). Llama 3 reference model code (models/llama3/model.py).
- Quesnelle, J., Peng, B., Fan, H., & Shippole, E. (2023). YaRN reference implementation.

23

Image credits

- Dürer, A. (1525). Tall graduated scale with a fan of lines and a small dial, a construction for proportional measure. *Underweysung der Messung*. SLUB Dresden. Public domain.

Answers

A1 · Slow pairs leave the trained range

- Illustrative: `b = 10,000`, `d_r = 64`
- Every position still gets an angle
- Slow pairs reach angles never seen in training

	Within 8K	At 32K
Slowest pair's largest angle	1.1 rad	4.4 rad
Pairs below one full turn	7 of 32	2 of 32

▼ Why serving past the training length is a distribution shift

Suppose a RoPE-based checkpoint was trained with contexts up to `L_0 = 8K` tokens and must now serve inputs of `L_t = 32K` tokens, without retraining from scratch. RoPE gives every position an angle and has no maximum position. So why not serve the 8K-trained checkpoint at 32K as it is?

Here `L_0` is the largest context represented in the relevant training stage. It can be smaller than the maximum position the architecture's configuration allows, so read `L_0` from the model's training report or training records.

Recall from Week 3 that RoPE splits each head's `d_r` rotated coordinates into `d_r/2` pairs. Pair `j` turns by `ω_j = b^(-2j/d_r)` radians per token, where `b` is RoPE's base. The first pairs turn fast and the last ones slowly. A pair's angle grows linearly with position and has no upper limit (Su et al., 2024). So a 32K input is valid, and the model accepts it. What changes is what the model sees. Past `L_0`, the input holds absolute indices that the training stage never contained, and relative offsets of larger magnitude than any it contained.

A worked example shows where the change is largest. Take the illustrative values `b = 10,000` and `d_r = 64`, so each head has 32 rotary pairs. The fastest pair turns by 1 rad per token and completes about 1,300 full turns within 8K tokens, so training has shown it every angle many times. The slowest pair turns by only about $1.33 \cdot 10^{-4}$ rad per token. Its wavelength, the number of tokens one full turn takes, is about 47,100 tokens. Within 8K its angle reaches only about 1.1 rad (63°). At 32K it reaches about 4.4 rad. Seven of the 32 pairs never complete one turn within 8K tokens. At 32K, 2 of the 32 still stay below one turn.

At 32K the slowest pairs take angles that training never produced. This is the argument YaRN makes for treating the slow pairs differently from the fast ones (Peng et al., 2024). Real models differ: a higher base, such as the 500,000 of Llama 3, leaves more pairs below one turn within 8K.

How badly a model fails on such inputs depends on the model, the task, the length and where the evidence appears in the input. So measure how quality changes with input length in your own setting instead of assuming a cliff at some fixed length.

This chapter is required reading. It has no submission, and later lectures do not depend on it. It ends with three reflection questions, which the Week 4 tutorial discusses, so read it before that tutorial and prepare a short answer to each. The chapter is about extending a RoPE-based checkpoint. NoPE and hybrid positional designs, which Week 3 covers, are the alternative path. Some recent models avoid the problem by leaving RoPE out of their long-range layers. The global attention layers of MAI-Thinking-1, which attend over the whole context, use no positional encoding (Microsoft AI Team, 2026) . OLMo Hybrid removes RoPE for its long-context stage (Merrill et al., 2026). It uses DroPE, a method that drops the positional embeddings of a pretrained model (Gelberg et al., 2026).

Back to the question

A2 · PI shrinks every pair; NTK spares the fastest

Adjacent-angle ratio	PI	Static NTK-aware
Fastest pair, `j = 0`	0.25	1
Slowest pair, `j = 31`	0.25	0.25

- PI: one factor for every pair
- NTK: the fastest pair keeps its angle

▼ Two ratios at the two ends

The question asks for the adjacent-angle ratio `ω'_j/ω_j` that PI and static NTK-aware scaling give the fastest pair, `j = 0`, and the slowest pair, `j = 31`, when `ρ = 4` and `d_r = 64`.

PI divides every adjacent angle by `ρ`, so its ratio is 1/4 = 0.25 at both ends, and at every pair between them.

For static NTK-aware scaling, substitute into the ratio `ρ^(-2j/(d_r-2))`, whose exponent here is `-2j/62`. At `j = 0` the ratio is `4^0 = 1`. At `j = 31` it is `4^(-62/62) = 0.25`. A pair in the middle shows the shape between the ends: at `j = 16` the ratio is `4^(-32/62) ≈ 0.489`, almost twice PI's 0.25.

Both methods bring the slowest pair to `1/ρ`. They differ in what they do to the fast pairs: PI slows them by the full factor, while NTK-aware scaling leaves the fastest pair unchanged and slows its neighbours only a little.

Back to the question

A3 · At 8K, YaRN keeps 13 pairs and interpolates 7

- Pairs 0–12: over 32 turns, kept
- Pairs 25–31: under one turn, interpolated
- Pairs 13–24: blended
- `j = 16`: `r = 13.04`, `γ = 0.388`

▼ **Walking the ramp across 32 pairs**

The question applies YaRN's ramp for the Llama family, `α = 1` and `β = 32`, to the illustrative values `L_0 = 8K` (8,192 tokens), `b = 10,000` and `d_r = 64`. Which pairs does YaRN keep, and what is `γ` for pair `j = 16`?

Combine Formula (4) with `ω_j = b^(-2j/d_r)` to get `r_j = L_0·ω_j/(2π) = 8,192·10,000^(-j/32)/(2π)`. The count falls by the factor `10,000^(1/32) ≈ 1.33` from each pair to the next. The fastest pair turns about 1,304 times. Pair 12 still turns about 41 times and pair 13 about 31 times, so pairs 0 to 12 have `r_j > 32`. Their weight is `γ_j = 1`, and they keep their original frequency. At the other end, pair 24 turns about 1.3 times and pair 25 about 0.98 times, so pairs 25 to 31 have `r_j < 1`. Their weight is `γ_j = 0`, and they are interpolated as under PI. These are the 7 pairs that slide 1 found never complete one turn within 8K tokens: with `α = 1`, the ramp's lower boundary is one full turn. Pairs 13 to 24 fall between the two boundaries and are blended.

For `j = 16`, the frequency is `ω_16 = 10,000^(-1/2) = 0.01` rad per token. Its wavelength is `λ_16 = 2π/0.01 ≈ 628.3` tokens per cycle, so `r_16 = 8,192/628.3 ≈ 13.04`. This count lies between 1 and 32, and the ramp gives `γ_16 = (13.04 - 1)/(32 - 1) ≈ 0.388`. With `ρ = 4`, PI's frequency for this pair is `0.01/4 = 0.0025`, and Formula (5) gives `ω'_16 = (1 - 0.388)·0.0025 + 0.388·0.01 ≈ 0.00541`. The blended frequency lies between PI's 0.0025 and the original 0.01.

Back to the question

A4 · Scaling only rotated coordinates gives 5.59

$$a^2\ell_r + \ell_u \approx 1.2965 \cdot 2 + 3 \approx 5.593$$

- Whole logit scaled instead: 6.482
- DeepSeek-V3's code scales the whole logit (DeepSeek-AI, 2025)

▼ **Partial rotary changes what scaling touches**

The question takes `p = 4`, for which `a ≈ 1.1386`, and a head whose normalized logit has a rotated part `ℓ_r = 2` and an unrotated part `ℓ_u = 3`. What is the new logit if only the rotated coordinates of Q and K are scaled by `a`?

Start from the normalized logit `ℓ = ℓ_r + ℓ_u`. The dot product of a query and a key is a sum over the head's `D_h` coordinates. `ℓ_r` collects the terms of the `d_r` rotated coordinates, `ℓ_u` collects those of the `D_h - d_r` unrotated ones, and both carry the factor `1/√D_h`. Scaling one coordinate of the query and the same coordinate of the key by `a` multiplies their product by `a²`. With full rotary there are no unrotated coordinates, so scaling Q and K by `a` gives `ℓ' = a²ℓ`. With partial rotary, scaling only the rotated coordinates gives `ℓ' = a²ℓ_r + ℓ_u`.

For `p = 4`, `a = 0.1 ln 4 + 1 ≈ 1.1386` and `a² ≈ 1.2965`. Scaling only the rotated part gives `1.2965·2 + 3 ≈ 5.593`. Scaling the whole logit instead gives `1.2965·(2 + 3) ≈ 6.482`. The values of `ℓ_r` and `ℓ_u` are illustrative. The two results come from two implementation choices: scale only the rotated part, or scale the whole logit. Before you reuse a YaRN implementation, check which of the two it scales.

DeepSeek-V3 is a real model that makes the whole-logit choice. It uses multi-head latent attention (MLA), which Week 4 covers. In MLA, each head's query and key have a 128-coordinate part without RoPE and a 64-coordinate part that carries RoPE, and the key's RoPE part is shared by all heads. Here `d_r = 64` of `D_h = 192` coordinates rotate. YaRN's frequency remap acts only on the RoPE part (DeepSeek-AI et al., 2024). Yet when the maximum sequence length exceeds the original, the reference code multiplies the softmax scale `1/√D_h` by `a²`, so it scales every logit (DeepSeek-AI, 2025). The part without RoPE is scaled too.

Back to the question

A5 · Flat perplexity and retrieval are not enough

- A task that needs distant evidence
- 8K regression, latency and memory
- Several prompts, documents and seeds
- Tokenizer and decoding details recorded
- A threshold set before the run

▼ What the two passing checks leave open

The question describes an extension from 8K to 32K whose perplexity stays flat up to 32K and whose retrieval passes with the evidence early, in the middle and late. What is still missing before you call the extension done?

Flat perplexity says that the model predicts the next token about as well at 32K as at shorter lengths. Passing retrieval says that it can find a string anywhere in the input. These are the first two rows of the matrix, and five rows remain open.

The distant-evidence row needs a task whose answer depends on using that evidence, such as aggregating facts from both ends of a document. The original-window and serving row asks whether quality at 8K regressed and what serving at 32K costs in latency and memory. One run on one set of inputs cannot show that the result holds across prompts, documents and seeds, which the robustness row asks for. Others can reproduce the result only if the tokenizer, the prompt format and the decoding details are recorded, as the reproducibility row requires. Without a threshold chosen before the run, the numbers describe the model but decide nothing, and the decision row supplies that threshold.

Back to the question

A6 · Remap the angles, train for use, measure both

- Every position still gets an angle
- Slow pairs leave the trained range
- Remapping brings the angles back
- Long-data training improves distant-token use
- Only the evaluation matrix decides

▼ The answer in five steps

So why not serve an 8K-trained model at 32K? We answer in five steps.

First, the model accepts the input, because RoPE gives every position an angle (The problem).

Second, the slow pairs then reach angles that the training stage never produced, and the model meets inputs unlike any it was trained on (The problem).

Third, a remapping brings the angles back into the trained range. PI divides every frequency by `ρ`, and static NTK-aware scaling raises the base and spares the fastest pair (PI and NTK). YaRN blends PI's frequencies with the original ones by rotation count and adds an attention scaling, and LongRoPE searches a rescale factor for each pair (YaRN and LongRoPE).

Fourth, a remapping changes only the coordinates. In the reports, training on long data improved the model's use of distant tokens (Training). Four of the five recipes on slide 13 train the weights for length, and every row adds a remapping, a high base, or both.

Fifth, only the evaluation matrix shows whether an extension worked, measured on the extended model against a threshold set in advance (Evaluation).

Back to the question